Doctorly — Senior .NET Interview Prep

Role stack: C#, .NET Core, ASP.NET Core, EF Core · Git/team workflow, code reviews, pair programming **Nice to have:** DDD/CQRS/SOLID · TDD/BDD · PostgreSQL · GitLab CI/CD **Company:** Berlin-based cloud PVS (Praxisverwaltungssystem) for German medical practices. Real-time multi-user editing of patient records; zero-downtime migration from legacy practice software; regulated, audit-heavy, sensitive-data domain (GDPR, Telematikinfrastruktur, EBM/GOÄ billing).

---

1. Your narrative (the 2-minute opener)

Have this rehearsed so you don't improvise it cold.

> *"I'm a senior .NET developer working on a private-markets transfer-agency platform — C#, ASP.NET Core, EF Core — where the core of the work is regulated workflows: KYC, audit trails, and data that's legally sensitive and can't just be mutated freely. So I'm used to building systems where correctness, traceability, and concurrency matter more than raw feature velocity. That's why doctorly interested me — patient data and billing are the same shape of problem: regulated, sensitive, multi-user. I've also been deliberately deepening the architecture side — DDD, CQRS with separate read models, system design — which is where I want to grow."*

Why this works: it maps your regulated-domain experience onto theirs (most candidates can't), and it names the architecture growth honestly rather than overclaiming.

---

2. C# / .NET Core — model answers

async/await — what's actually happening? `await` doesn't create a thread. It registers a continuation and releases the current thread back to the pool until the awaited operation completes. For I/O-bound work (DB, HTTP), this is what lets a small thread pool serve many concurrent requests.

Why not `.Result` or `.Wait()`? Blocking on an async call can deadlock — classically in contexts with a synchronization context where the continuation needs the thread you're blocking. Even in ASP.NET Core (no sync context) it wastes a pool thread and defeats the point of async. Rule: async all the way down.

`CancellationToken` — why pass it everywhere? So work stops when the caller goes away (request aborted, timeout). In a medical app a user navigating off a slow patient query should free the DB connection, not keep it pinned. EF Core query/save methods take a token — pass the request's `HttpContext.RequestAborted` through.

DI lifetimes — and the trap: - *Transient*: new instance every resolve. - *Scoped*: one per request (DbContext is scoped). - *Singleton*: one for app lifetime. The classic bug: injecting a **scoped DbContext into a singleton**. The singleton captures one DbContext forever — it's not thread-safe and you get cross-request state corruption. Fix: inject `IServiceScopeFactory` (or `IDbContextFactory<T>`) and create a scope per unit of work.

`IEnumerable` vs `IQueryable` (they'll likely tie this to EF Core): `IQueryable` builds an expression tree the provider translates to SQL — filtering happens **in the database**. `IEnumerable` runs LINQ **in memory**. The senior failure mode: calling `.AsEnumerable()` / `.ToList()` too early, then filtering — you've pulled the whole table into memory. Know exactly where the query "materialises."

Value vs reference types / records: Structs are copied by value, live on the stack (usually), good for small immutable data. Classes are references. `record` gives you value-based equality and `with`-expressions — useful for DTOs, value objects, and CQRS commands/queries.

---

3. ASP.NET Core specifics

- **Middleware pipeline:** ordered chain; each middleware can short-circuit. Order matters — e.g. `UseAuthentication` before `UseAuthorization`, exception handling near the top. Be ready to explain what you'd put in custom middleware (correlation IDs, request logging, audit context).
- **Model binding + validation:** binding maps request to DTOs; validation via data annotations or FluentValidation. For a regulated app, validate at the boundary and keep domain invariants in the domain.
- **Auth:** know the difference between authentication (who are you) and authorization (what can you do), and roughly how JWT bearer auth flows. You won't need German TI specifics, but knowing the app sits behind strong auth + audit logging signals domain awareness.

---

4. EF Core — over-prepare this (it's core to stack AND product)

Optimistic concurrency — your highest-value answer

This is *the* answer to "how do two MFAs edit the same patient record safely?"

- Add a concurrency token — in PostgreSQL the idiom is `xmin` (system column) mapped via `.IsRowVersion()` / `UseXminAsConcurrencyToken()`; in SQL Server it's `rowversion`/`[Timestamp]`.
- On `SaveChanges`, EF adds the original token value to the `WHERE` clause. If another transaction changed the row, 0 rows match → `DbUpdateConcurrencyException`.
- You then resolve: **store wins**, **client wins**, or **merge** — for clinical data you usually reload, show the user what changed, and let them re-apply rather than silently overwrite.

Say this explicitly: "*For patient records I'd default to detecting the conflict and surfacing it, not last-write-wins — silently dropping a clinician's edit is a safety problem*." That sentence is the one they'll remember.

Other EF Core must-knows

- **Change tracking & `AsNoTracking()`:** reads that don't update should be no-tracking — less memory, faster. Track only when you intend to write.

- **N+1 problem:** lazy loading inside a loop fires a query per row. Fix with `Include`/`ThenInclude` or, better, **project** to a DTO with `Select` so you fetch only needed columns. Mention `AsSplitQuery()` to avoid cartesian explosion with multiple `Includes`.
- **Migrations:** code-first, reviewed like code, applied in CI. Know that destructive migrations on a live medical DB need care (additive first, backfill, then remove).
- **Transactions / Unit of Work:** `SaveChanges` is a transaction; for multi-aggregate operations use an explicit transaction or the outbox pattern for reliability.

Testing EF Core (likely follow-up)

The strong answer: **Testcontainers running real PostgreSQL**, not the in-memory provider. In-memory doesn't enforce relational constraints, transactions, or concurrency — so it gives false confidence. Use it only for trivial cases; integration tests hit a real Postgres in a container.

---

5. DDD / CQRS / SOLID

Aggregate / Entity / Value Object (plain definitions): - *Entity*: has identity that persists over time (a Patient). - *Value object*: defined by its values, immutable, no identity (an Address, a billing Money amount). - *Aggregate*: a cluster with one root; you only modify the inside through the root, which guards invariants. (A patient record with its consultations might be an aggregate — you don't edit a consultation outside its patient context.)

CQRS — and when NOT to: Separate the write model (commands, enforce invariants) from the read model (queries, shaped for the screen). It shines when reads and writes have very different shapes/scale — *which is exactly your document-tree refactor: flat write model, hierarchical read model.* Use that as your example. **When not to:** simple CRUD with matching read/write shapes — CQRS there is just ceremony and two models to keep in sync.

SOLID — give real examples, not definitions: - *S*: a handler that both validates AND emails AND persists is doing too much — split it. - *O*: add a new billing rule via a new strategy class, not by editing a switch statement. - *L*: a subclass that throws on a method the base supports breaks substitutability. - *I*: don't force a class to implement a fat interface it half-uses. - *D*: depend on `IPatientRepository`, not a concrete EF class — that's also what makes it testable.

---

6. Testing — TDD / BDD

- **TDD:** red → green → refactor. Sell it as design pressure, not just coverage — writing the test first forces a usable API and small units.
- **Test pyramid:** many fast unit tests, fewer integration tests (real Postgres via Testcontainers), very few end-to-end.
- **BDD:** Reqnroll (the maintained SpecFlow successor) with Gherkin — valuable on a *regulated* product because Given/When/Then specs double as living, business-readable documentation of compliance rules.
- **Mocking:** Moq or NSubstitute for collaborators behind interfaces; don't mock what you don't own / don't mock the DB — integration-test it.

7. PostgreSQL (if your current shop is SQL Server, know the delta)

- Driver is **Npgsql**; provider `Npgsql.EntityFrameworkCore.PostgreSQL`.
- Concurrency token idiom is `xmin`, not `rowversion`.
- Rich native types: `jsonb` (queryable JSON — handy for flexible clinical metadata), arrays, real `text`, `uuid`.
- Case sensitivity and identifier quoting differ from SQL Server; `citext`/`ILIKE` for case-insensitive matching.
- It's MVCC-based — readers don't block writers — worth a sentence if concurrency comes up.

If you haven't used Postgres in production, say so plainly and pivot to the transferable EF Core knowledge plus the specific deltas above. Honesty + "here's what I'd need to learn" reads as senior.

---

8. GitLab CI/CD

- Pipeline defined in `.gitlab-ci.yml`: **stages** (build → test → deploy) made of **jobs** run by **runners**.
- Key concepts to name: caching dependencies, artifacts between stages, environment-specific deploys, manual approval gates for production, secrets via CI/CD variables (never in the repo).
- For a medical product: tests + migrations gated in CI, protected branches, merge-request pipelines, and review approval before merge. Tie this back to **code review** — they named it explicitly.

---

9. System design walkthrough — "design shared patient-record editing"

Rehearse this out loud once. A clean structure:

1. **Clarify:** how many concurrent editors? Field-level or whole-record? Need live cursors or just safe saves?
2. **Concurrency model:** optimistic concurrency with a version token (section 4). On conflict, reload and surface the diff — never silently overwrite clinical data.
3. **Live updates:** SignalR to push "another user is editing / has saved" to connected clients.
4. **Granularity:** consider section/field-level locking or versioning so two people editing different parts of the record don't collide.
5. **Audit:** every change writes an immutable audit entry (who, what, when, before/after) — non-negotiable for medical data.
6. **Soft deletes:** never hard-delete clinical records; flag + retain.
7. **Trade-offs:** optimistic (good UX, rare conflicts) vs pessimistic locking (blocks the doctor — usually wrong here).

---

10. Messaging & async communication — decision tree

A clean way to reason out loud if they ask "how would services talk to each other" or "when would you add a message broker." Walk it top-down:

```
Do I need asynchronous communication?
├─ NO  -> HTTP / REST / gRPC (synchronous request/response)
└─ YES -> Do the volume and complexity justify a broker?
          ├─ NO  -> Postgres / Redis as a simple queue
          └─ YES -> Do I need to keep messages after they're processed?
                    ├─ NO  -> Am I on AWS?
                    │         ├─ YES -> SQS
                    │         └─ NO  -> RabbitMQ
                    └─ YES -> Could multiple systems need to read
                              these messages now or in the future?
                              ├─ NO  -> RabbitMQ / SQS
                              └─ YES -> Kafka (durable, replayable log)
```

The key distinctions to be able to articulate: - **Sync vs async:** use sync (REST/gRPC) when the caller needs the answer now; use async when the work can happen later and you want to decouple producer from consumer. - **Queue vs log:** RabbitMQ/SQS deliver a message to *a* consumer and it's gone once acked — good for work distribution. Kafka is a durable, replayable log many consumers can read independently — good when several systems (or a future one) need the same event stream. - **Don't over-engineer:** a broker is operational overhead. If the volume is low, a Postgres/Redis-backed queue is often enough — saying this signals seniority more than reaching for Kafka.

Tie it to doctorly: synchronous REST for the practice-management API; **SignalR** for live patient-record updates (not a broker — it's server→client push); and a **broker** for genuinely async work — billing runs, the legacy-data migration pipeline, sending notifications. Kafka only if multiple services need to consume the same clinical/billing event stream durably (e.g. audit, analytics, and billing all reading "consultation recorded"). For a single practice's volume, RabbitMQ/SQS is the more honest default.

---

11. Behavioural — STAR answers from your real experience

Prepare these as stories, not bullet points:

- **Code-review philosophy:** what you block on (correctness, security, missing tests, unclear domain naming) vs what you let go as a comment (style preferences). How you disagree: ask a question rather than assert, and defer to the owner on reversible decisions.
- **A time you changed your mind in review/pairing:** shows ego-free collaboration — exactly what "pair programming" in the JD is screening for.
- **A regulated/complex workflow you modelled:** your KYC triage work — the constraints, how you kept the domain logic clean and audited.
- **A refactor under constraints:** the flat→hierarchical document-tree migration with read models — why, the risk, how you de-risked it.
- **Disagreement with a decision you then committed to:** "disagree and commit" — senior signal.

---

12. Questions to ask them (asking good ones is part of the eval)

- How do you currently handle concurrent edits to a patient record, and where does that model show strain?
- What does the TI / regulatory side mean for day-to-day engineering — how much does it constrain the codebase?
- How is the team split — feature teams, bounded contexts, a modular monolith or services?
- What does your code-review and pairing culture actually look like day to day?
- Where's the legacy migration tooling now — is that a separate system?
- What does the test/deploy pipeline look like, and how often do you ship?

---

13. Final checklist

- [ ] 2-minute opener rehearsed out loud
- [ ] Optimistic-concurrency answer can be drawn on a whiteboard
- [ ] Scoped-DbContext-in-singleton trap memorised
- [ ] N+1 + projection explained with a concrete query
- [ ] CQRS "when not to" + your document-tree example ready
- [ ] Async-communication decision tree memorised (broker only when justified)
- [ ] Testcontainers-vs-in-memory line ready
- [ ] 3 behavioural stories in STAR form
- [ ] 4 questions to ask them
- [ ] Postgres deltas if SQL Server is your background